

Adaptive Variable-Resolution 2.5D LiDAR Mapping

Condensed Build Plan & Technology Study Guide — Smart India Hackathon

Foveated perception for autonomous vehicles: high spatial detail near the vehicle, progressively coarser detail at distance, without sacrificing safety-critical near-field accuracy.

1. Pipeline at a Glance

Raw LiDAR → preprocessing & ego-motion → semantic perception → terrain/traversability → dynamic-object detection → adaptive radial grid → temporal fusion → visualization → benchmarking.

Stage	What to build	Primary technologies
0. Preprocessing + ego-motion	Filtering, coordinate transforms, pose compensation	Open3D, NumPy, ROS 2 tf2 / KITTI odometry GT
1. Semantic perception	Point/voxel classification (12-class taxonomy)	PyTorch, PointNet++, (later) Minkowski Engine
2. Terrain & traversability	Ground plane, slope, roughness, safe/caution/non-drivable score	Open3D, RANSAC, NumPy
3. Dynamic objects	Clustering, association, Kalman tracking, static/dynamic label	scikit-learn (DBSCAN), filterpy / custom Kalman
4. Adaptive grid	Radial hash-grid, 5/10/25/50 cm bands, deterministic point→cell mapping	Custom Python/C++ engine
5. Temporal fusion	Bayesian occupancy update, decay, dynamic-cell exclusion	Custom (log-odds update)
6. Visualization	Semantic / elevation / traversability / resolution / performance views	Streamlit or React, Open3D, RViz (optional)
7. Deployment/optimization	Latency & memory profiling, inference speed-up	PyTorch Profiler, TensorRT

Golden rule: keep the perception model fixed-resolution throughout. All distance-dependent complexity lives in the mapping layer. This isolates your core novelty (the adaptive grid) from model-training risk.

2. Dataset Strategy

Track A — SemanticKITTI (primary, start Day 1): point-wise semantic labels for training/validating segmentation. Collapse the ~19 classes into your 12-class taxonomy (drivable, non-drivable, road, sidewalk, curb, wall, pole, vehicle, pedestrian, cyclist, vegetation, unknown) with a single remap table, unit-tested before any training begins.

Track B — nuScenes (parallel, start Day 3–4): 3D object boxes and temporal annotations for dynamic-object tracking and velocity ground truth. Run as an independent track rather than sequentially — the two datasets have different sensor geometry (64-beam vs 32-beam) and class taxonomies, so reconciling them fully is not worth the time; use SemanticKITTI for segmentation quality and nuScenes only for tracking/velocity validation.

Traversability ground truth: neither dataset labels "safe / caution / non-drivable" directly. Derive pseudo-labels from SemanticKITTI terrain classes plus your own slope/roughness rules, and manually spot-check ~50 frames. State this limitation explicitly in your report.

Optional/later: KITTI (classical baselines), Waymo Open Dataset (large-scale evaluation, stretch goal only).

3. Adaptive 2.5D Grid — Core Specification

Range	Resolution	Primary use
0–10 m	5 cm	Collision-critical geometry, curbs, potholes, pedestrians
10–25 m	10 cm	Near-field planning and object structure
25–50 m	25 cm	Medium-range awareness
50–100 m	50 cm	Long-range awareness at low memory cost

Data structure: hash map keyed by `(ring_id, cell_row, cell_col)`, not a dense array — cells materialize only on first point insert, which is the actual mechanism of the memory saving. For each point: compute range $r = \sqrt{x^2+y^2}$, select exactly one ring via half-open intervals `[low, high)`, compute integer cell indices via `floor(x/res)`, `floor(y/res)`. Choose resolutions so each coarser band is an integer multiple of the finest (5 → 10 → 25 → 50), so ring boundaries tile without seams. Unit-test every ring boundary explicitly ($r = 9.999, 10.000, 10.001$ m).

Cell payload: elevation mean/min/max/variance, semantic class + confidence, occupancy probability, dynamic probability, point count, timestamp, velocity (if dynamic).

Verified math: a uniform 100×100 m grid at 5 cm has exactly 4,000,000 cells. A rigorous ring-area calculation for the four-band schedule gives ≈480,000 cells (circular footprint) — about **12% of uniform-5cm**, an ~8.3× reduction. Report your own measured cell count from the running implementation as the final number; use this figure only as the predicted target.

4. Implementation Sequence (10-day arc)

Day	Task	Method / tooling	Done-when
1	Load, filter, visualize; dataset loader + class remap	Open3D, SemanticKITTI API	Point clouds correctly loaded/transformed; remap table tested
1–3	Perception baseline	PyTorch + PointNet++ (or pretrained SalsaNext/RangeNet++)	Semantic predictions on held-out frames
3	Uniform 5 cm map (= Baseline A)	NumPy + Open3D	Working dense 2.5D baseline, kept for final comparison
3–6	Adaptive grid engine CORE	Custom radial hash-grid	Zero point loss/duplication at every boundary test
4–6	Terrain/traversability (parallel)	RANSAC ground plane, slope/roughness rules	Safe/caution/non-drivable layer producing sane maps
5–7	Dynamics (parallel)	DBSCAN + nearest-centroid + Kalman filter	Static vs dynamic labels stable across a test sequence
6–8	Temporal fusion	Log-odds Bayesian update, decay, pruning	Dynamic cells excluded from static accumulation, provably
7–9	Dashboard (parallel with fusion)	Streamlit/React, uniform-vs-adaptive toggle	All 5 required views live and toggleable
8	Sparse model swap (stretch, only if ahead)	Minkowski Engine	Latency/accuracy comparison vs PointNet++
9	Benchmarking	Custom eval suite	Baseline A / B / Adaptive compared, distance-banded
10	Demo + report	—	Video recorded, report written

Stages 3–6 (grid) and 4–7 (terrain, dynamics) run **in parallel** across team members — a linear reading of this table wastes people. Assign one owner per stage from Day 3 onward.

5. Evaluation Plan

- **Semantic segmentation:** mIoU, precision, recall, F1, confusion matrix
 - **Terrain:** IoU/F1 + false-drivable rate (safety-critical headline metric)
 - **Dynamic objects:** precision/recall, velocity error where GT supports it
- **Mapping:** active cell count, projection latency, memory (RAM+VRAM)
 - **Runtime:** preprocessing / inference / projection / fusion / end-to-end latency, FPS
 - **Every metric above, broken out by band:** 0–10, 10–25, 25–50, 50–100 m

Ablation: compare Uniform-5cm (Baseline A) vs Uniform-50cm (Baseline B) vs Adaptive, same sequence, same hardware. Target result: near-field metrics for Adaptive ≈ Baseline A; far-field memory/compute for Adaptive ≈ Baseline B.

Core research question: *Can an autonomous vehicle preserve safety-critical near-field spatial awareness while reducing the memory and computational cost of large-range LiDAR mapping?*

6. Technologies to Study

Organized by pipeline stage. **CORE** = essential, study first. Study the reading/docs listed for each before writing code against it.

6.1 Point Cloud Fundamentals **CORE**

Technology	What to study	Resource
Open3D	I/O, filtering, downsampling, KD-trees, coordinate transforms, visualization	open3d.org/docs/latest/tutorial/Basic/pointcloud.html github.com/isl-org/Open3D
NumPy	Vectorized point-cloud math; avoid per-point Python loops for anything in the hot path	numpy.org/doc/stable

6.2 Deep Learning Framework **CORE**

PyTorch	Tensors, autograd, DataLoader for point-cloud batches (variable point count per frame), GPU execution, checkpoint loading for pretrained models	docs.pytorch.org/docs/stable
---------	---	---

6.3 Semantic Segmentation Architectures

Model	Why study it	Resource
PointNet CORE	Foundational — permutation-invariant point processing; read before PointNet++	arxiv.org/abs/1612.00593
PointNet++ CORE	Your MVP baseline segmentation model — hierarchical local feature grouping via farthest-point sampling	arxiv.org/abs/1706.02413
Cylinder3D	Cylindrical voxel partition matched to radial/distance-based grids — closest prior art to your own grid design; study for architectural intuition even if not implemented	arxiv.org/abs/2008.01550 github.com/xinge008/Cylinder3D
Minkowski Engine (4D sparse conv)	Sparse voxel convolution — use to replace PointNet++ once MVP is stable, for latency comparison	nvidia.github.io/MinkowskiEngine arxiv.org/abs/1904.08755
RangeNet++	Range-image projection — fastest inference option; useful fallback if latency is the bottleneck	github.com/PRBonn/rangenet_lib

6.4 Datasets

Dataset	What to study	Resource
SemanticKITTI CORE	Label taxonomy, sequence/scan structure, the API's loading and evaluation tools	semantic-kitti.org github.com/PRBonn/semantic-kitti-api
nuScenes	3D box format, tracking annotations, devkit's data-loading and visualization utilities	nusenes.org/nusenes github.com/nutonomy/nusenes-devkit
KITTI / Waymo Open (optional)	Only if time permits — classical baselines / large-scale stress test	waymo.com/open

6.5 Terrain, Elevation & Multi-Resolution Mapping **CORE**

Concept / library	What to study	Resource
RANSAC plane fitting	Robust ground-plane estimation per local patch, not one global plane	Open3D segment_plane docs
GridMap (Fankhauser & Hutter)	Multi-layer elevation/traversability map design — direct ancestor of your cell schema	github.com/ANYbotics/grid_map
OctoMap	Canonical multi-resolution occupancy structure; conceptual justification for variable resolution	octomap.github.io
Quadtree/adaptive occupancy grids	Prior art on adaptive-resolution grid mapping (accuracy-driven, vs. your distance-driven approach) — read to correctly frame your novelty	Search: "variable resolution occupancy grid quadtree"

6.6 Dynamic Object Detection & Tracking

Technology	What to study	Resource
DBSCAN / Euclidean clustering	Instance clustering of obstacle-class points	scikit-learn.org (<code>sklearn.cluster.DBSCAN</code>)
Kalman filter (constant-velocity model)	Per-object tracking across frames; state = position + velocity	filterpy docs , or implement from first principles
Moving Object Segmentation (Chen et al.)	Distinguishing moving vs. parked cars — directly matches your static/dynamic requirement	arxiv.org/abs/2105.08971

6.7 Robotics Integration & Visualization

Technology	What to study	Resource
ROS 2	Topics, tf2 for pose/ego-motion transforms, bag playback for recorded LiDAR data	docs.ros.org
RViz	Point cloud, marker, and occupancy grid display types (optional if using a custom web dashboard instead)	docs.ros.org (<code>rviz2</code> package)
Streamlit	Fast Python web dashboard — live semantic/elevation/traversability/resolution/performance views	docs.streamlit.io

6.8 Performance & Deployment

Technology	What to study	Resource
PyTorch Profiler	Per-stage latency breakdown (preprocessing/inference/projection/fusion), GPU memory tracing	docs.pytorch.org/tutorials (profiler recipe)
TensorRT	Model optimization/quantization for deployment-grade inference latency — stretch goal, only after MVP is stable	docs.nvidia.com/deeplearning/tensorrt/latest

Study order recommendation: 6.1 → 6.2 → 6.3 (PointNet, PointNet++ only) → 6.4 (SemanticKITTI) → 6.5 in full → 6.6 → 6.7 → 6.8. Treat Cylinder3D, Minkowski Engine, RangeNet++, and nuScenes as concurrent/parallel reading once the MVP loop (load → segment → uniform map) is working — don't block Day 1–3 progress on reading the full architecture survey first.

Condensed from the full SIH resource & architecture plan. Team rule: build before you optimize — prove LiDAR loading, then segmentation, then a uniform 2.5D map, then replace only the mapper with the adaptive engine.